



Manit Hub — The Build Story

How a campus super-app for MANIT Bhopal was actually built — in order, decision by decision, mistake by mistake. What was planned, what was chosen and why, what broke, and how every problem was fixed.

Covers the full development window: **19 May 2026** → **14 July 2026** · 78 commits

Web: manithub-samayjainbm.vercel.app · API: manit-hub.onrender.com · Android APK via Capacitor

READ IN ONE SITTING — THE CHAPTERS

- | | |
|---|---|
| 1. The idea — why build this at all | 11. Leaving serverless — the move to Render |
| 2. Deciding the process before the code | 12. The database design mistake (E11000) |
| 3. Choosing the stack (and what was rejected) | 13. Production rewrites the storage plan |
| 4. Designing the database | 14. The login that would not stay |
| 5. Writing the backend | 15. Going native — the Android app |
| 6. Day one in production — the deployment war | 16. The restructures |
| 7. Maps, a name, and a routing lesson | 17. Security hardening & email verification |
| 8. Making it feel alive — seed data & notifications | 18. The frontend, as it ended up |
| 9. The feature blitz | 19. Lessons learned |
| 10. Real-time chat — the three hardest bugs | 20. Appendix — the full timeline |

1 · The idea — why build this at all

Every project should start from an irritation, not a technology. Manit Hub started from campus life at MANIT Bhopal being scattered across fifty WhatsApp groups.

Selling a used textbook meant posting in five groups and hoping. Finding notes before an exam meant begging seniors. Losing an ID card meant a broadcast message nobody read. Checking whether you could safely bunk one more class meant manual arithmetic on the back of a notebook. None of these problems is hard individually — the pain is that each one lives in a different, unsearchable, unmoderated place.

So the idea was a **student super-app**: one verified, students-only space for the whole campus loop — a marketplace, a study-material vault, CGPA and attendance trackers, a timetable, anonymous confessions, lost & found, events, ride-sharing, a course Q&A forum, and real-time chat tying it all together. Two constraints were set on day one and never relaxed:

- **Verified community only.** Sign-up restricted to `@stu.manit.ac.in` emails, so every account maps to a real student. This single rule is what makes a marketplace, confessions and chat safe enough to run.
- **Multi-tenant from the start.** Even though only MANIT would use it initially, every record would be scoped to a `university`, so expanding to other NITs later would be a data question, not a rewrite.

2 · Deciding the process before the code

Before writing anything, the order of work was decided. The plan was deliberately boring — and it mostly survived contact with reality.

The process decided up front:

1. **List the features, then cut.** Write down everything campus life needs, then pick the smallest set that is already useful: auth, marketplace, chat, study groups. Everything else (CGPA, attendance, confessions, rides, forum...) would be layered on after the skeleton worked.
2. **Design the database first.** Get the entities and their relationships on paper before any endpoint exists, because in a document database the schema *is* the architecture. (This is also where the one real design mistake was made — Chapter 12.)
3. **Backend before frontend, per feature.** For each feature: Mongoose model → API endpoints → then the React screen. The frontend was never blocked guessing what the API would return.

4. **Deploy on day one.** Don't build for months and then discover production. Get the empty skeleton live immediately, and keep it live after every change. This decision paid for itself within hours (Chapter 6).
[LLD Singleton Principle](#)
5. **One person, so one convention everywhere.** A single response envelope, a single middleware chain, one file per endpoint — so that six weeks in, every corner of the codebase still looks the same.

Why this order and not "frontend first"?

A UI-first build produces beautiful screens wired to imaginary data, and the rework lands later at the worst time. Data-model-first means the hard questions (who owns a listing? what makes two chats "the same"? what is private vs campus-visible?) get answered when changing the answer costs nothing. The trade-off accepted: the app looks ugly for longer. That was fine — there were no users yet.

3 · Choosing the stack (and what was rejected)

Every technology was chosen against a named alternative. "Popular" was not a reason; "fits a solo student building a real-time campus app for free" was.

Layer	Chosen	Rejected	The actual reason
Language	JavaScript everywhere	Python/Django, Java/Spring	One language for API, web and socket code. No context-switching, one npm ecosystem, and Socket.IO's best support is Node.
Database	MongoDB Atlas + Mongoose	PostgreSQL / MySQL	Campus features have wildly different shapes (a confession vs. a timetable entry vs. an offer). Document schemas iterate fastest when requirements change weekly; array fields (participants, images, RSVPs) fit naturally; Atlas has a free tier with zero ops work. The cost — fewer integrity guarantees — showed up exactly once (Chapter 12).
API framework	Express 5	NestJS, Fastify	Express is small enough to understand completely. Nest's module/decorator machinery adds structure a team needs but a solo dev pays for. Structure was instead imposed by a strict folder convention (Chapter 5).
Auth	JWT (bearer token)	Server sessions/cookies	The API serves a Vercel web app <i>and</i> a native Android app from a different origin — cookie sessions across two origins plus a WebView are painful. A stateless bearer token works identically in axios, Socket.IO auth, and native HTTP. Revocation was solved later with a <code>tokenVersion</code> claim (Chapter 17).
Real-time	Socket.IO	Raw WebSocket, polling	Rooms (one per conversation), automatic reconnection, and graceful fallbacks — all needed, none worth hand-writing. Polling was rejected because chat is the flagship "the app is alive" feature.
Frontend	React 19 + Vite 7	Next.js, CRA	The app lives behind a login — there is nothing to SEO except the landing page, so SSR buys nothing and complicates the Capacitor wrap. Vite's dev server and proxy made local dev instant. CRA was effectively deprecated.
Styling	Tailwind CSS 3	CSS modules, MUI	A custom design system (MANIT navy/crimson/gold, dark mode) was a goal — component libraries fight custom identity. Tailwind tokens made the theme consistent without a CSS architecture to maintain.
Mobile	Capacitor 8	React Native, Flutter	One codebase was non-negotiable for one developer. Capacitor ships the same built web app in a native shell — every web fix is instantly an Android fix. React Native would fork the UI layer; Flutter would fork the whole app. Trade-off accepted: WebView performance, mitigated by keeping the app light.
Media	Cloudinary (documents)	S3, local disk	Free tier, zero-config transforms, and streaming upload via one SDK call. Local disk dies on redeploy of a PaaS host.

			(Images later moved <i>out</i> of Cloudinary entirely — Chapter 13 tells that story.)
Hosting	Vercel (web) + Render (API)	Everything on Vercel	This was <i>not</i> the day-one plan. The API began on Vercel serverless and was forced off it by WebSockets — Chapter 11.

The honest summary

MERN was chosen not because it is fashionable but because every piece is free, JavaScript-only, and forgiving of changing requirements. Two of these choices later had to be corrected in public (serverless hosting for the API, Cloudinary for images) — and both corrections are documented in this story rather than hidden, because the corrections taught more than the choices did.

4 · Designing the database

The schema was drawn before the first endpoint. It ended up as 22 Mongoose models — and its two central decisions shaped everything downstream.

4.1 Decision 1 — every campus record carries a `university` field

Rather than assuming "this is a MANIT app," every shared collection — listings, study groups, documents, confessions, events, rides, questions, lost-&-found items, conversations — carries a required, indexed `university` reference. The user's university is embedded in their JWT at login, and middleware injects it into every query. No handler ever "remembers" to filter by campus; the filter is structural.

Why bake in multi-tenancy for a single-campus launch?

Because retrofitting tenancy is a rewrite: every query, every index, every socket room would need auditing. Adding it on day one cost one schema field and one middleware. It also doubles as a security boundary — even a buggy handler cannot leak another campus's data if the scope is applied before the handler runs.

4.2 Decision 2 — references between collections, embedding within them

Things that are queried independently and have their own lifecycle became collections joined by `ObjectId` references (User, Listing, Conversation, Message, Question, Answer...). Things that only ever live inside their parent were embedded as arrays or subdocuments — a listing's images, a ride's passenger list, a study group's member array, a confession's reactions. Personal academic data (CGPA records, attendance subjects, timetable entries) got private per-user collections that are never campus-visible.

4.3 The entity map, in one paragraph

A **University** has many **Users**. A User sells **Listings**, which receive **Offers** and spawn **Conversations** containing **Messages**. Users create and join **StudyGroups**, upload **Documents** to the vault, ask **Questions** that get **Answers**, post **Confessions** anonymously, report lost items as **LostFoundItems**, organise **Events** with RSVP lists, and post **Rides** with passenger arrays. **Reports** can point at any content type; **Notifications** and **DeviceTokens** hang off users; **AcademicRecord**, **AttendanceSubject**, and **TimetableEntry** are private academic data; **Friendship** models the social graph.

The mistake planted here

To guarantee "one conversation per buyer per listing," the Conversation model was given a compound **unique index on** `{ listingId, participants }` — where `participants` is an *array*. It looked like an obvious integrity constraint. It was actually a time bomb that would detonate five weeks later, in production, on the second chat any user ever opened. The full detonation and repair is Chapter 12 — it is the single best lesson in this document.

5 • Writing the backend

The backend is Express, but the real architecture is a filing convention: one folder per feature, one file per endpoint.

5.1 The one-file-per-endpoint pattern

Instead of fat controllers, the backend has 23 feature folders (`auth/`, `listings/`, `conversations/`, `forum/`, `offers/`...) and inside each, one small file per endpoint — `createListing.js`, `getListings.js`, `updateListing.js` — plus a `helpers.js` for shared bits. `app.js` just mounts one router per feature.

Why split it this small?

Three reasons. **Findability:** a bug report names a behaviour, and the behaviour names the file. **Blast radius:** editing "accept offer" cannot break "create offer" by accident. **Reviewability:** each file reads top-to-bottom in under a minute. The cost is more files — which matters to nobody with a fuzzy file finder.

5.2 The middleware gauntlet

Every authenticated request passes the same chain **before any handler runs:** `helmet` and `CORS` → `JSON parsing` → `auth` (verify JWT, load the user, reject banned accounts) → `universityScope` (pin the tenant filter, guard `:id` routes against cross-campus access) → the endpoint file → a shared error handler that returns a generic message in production so internals never leak. **Responses always use one envelope** — `{ success, message, data, meta }` — so the frontend has exactly one shape to parse.

5.3 Write-path discipline: field whitelists

Create/update handlers copy an explicit list of allowed fields from `req.body` instead of spreading the body into the model. That is what makes mass assignment impossible: no request can ever set `seller`, `university`, `isActive` or karma fields, because no code path reads them from user input.

6 · Day one in production — the deployment war

19 May 2026: the very first day produced fourteen commits, and most of them are deployment fixes. This was "deploy on day one" doing exactly its job — surfacing every environment assumption immediately.

The commit log of that single day reads like a diary of everything that differs between localhost and the cloud: *"fixed dependencies" → "added root route" → "update cors origin and frontend API URL" → "align vite api env name" → "fix mern deployment connection" → "mongodb atlas update" → "fix mongodb startup on vercel" → "fix vercel mongodb request gate" → "harden mongodb production config."*

Problem — MongoDB × serverless cold starts

Locally, you connect to Mongo once at boot and forget it. On Vercel, every serverless invocation may be a fresh process — naive code either reconnects on every request (slow, exhausts Atlas connections) or, worse, handles a request *before* the connection is ready and throws.

Fix — a connection gate

The DB module caches the connection promise and every request awaits that gate before touching a model: first request on a cold instance pays the connect cost once, all others reuse it. This "serverless-safe connection gate" survived even after the API later moved to a persistent host — it is harmless there and still guards restart races.

Problem — CORS and environment-variable naming

The deployed frontend called the deployed API and got CORS-blocked; separately, the frontend was reading an env var name that didn't match what was configured in the host (Vite only exposes variables prefixed `VITE_`). Both are invisible locally, where the dev proxy makes everything same-origin.

Fix

An explicit CORS allow-list driven by a `CLIENT_URL` env var, and one agreed name — `VITE_API_URL` — used everywhere. Locally, the Vite proxy forwards `/api` to the backend so development stays CORS-free entirely.

Lesson

Fourteen scrappy commits on day one were the cheapest education of the whole project. Every one of these failures would have happened anyway — the only choice was whether they happened against an empty app in hour six, or against real users in week six.

PART III — BUILDING THE PRODUCT

7 • Maps, a name, and a routing lesson

20 May 2026: the second day went to something visibly campus-specific — an accurate MANIT map — plus the project getting its real name.

The campus-maps feature went through four iterations in a day: an exact MANIT boundary polygon, then a KML-based boundary, then an embedded Google custom map, then a redesigned dashboard around it. The end state is 41 real locations — all 12 hostels, canteens, sports grounds, departments — each opening an embedded live map pin. The same day, the project was renamed to **Manit Hub**.

Problem — the classic SPA 404

On Netlify (the frontend's first home), refreshing any deep route like `/dashboard` returned a 404. A single-page app has exactly one real file — `index.html` — and the host doesn't know that `/dashboard` is a client-side route, not a file.

Fix

A catch-all rewrite rule sending every path to `/index.html` so React Router can take over. The same lesson had to be re-learned in Vercel's dialect (`vercel.json` rewrites) when the frontend later moved there. Every SPA developer hits this exactly once per hosting provider.

8 • Making it feel alive — seed data & notifications

Early June: before adding more features, the app got realistic demo data and its notification backbone — two unglamorous investments that paid off daily.

A seeding script (`backend/scripts/seed.js`) creates demo students, a dozen MANIT-flavoured listings, seven study groups, conversations and notifications — and is safe to re-run because it only resets its own data. This transformed development: every feature from here on was built and demoed against a campus that looked inhabited, and bugs like "the list looks wrong with 12 items" surfaced immediately instead of at launch.

Notifications were built as an **event-driven** system from the start: things that happen (a message, a group join, interest in your listing, an accepted offer) create notification documents, which the bell icon queries — rather than screens polling for "what changed?". This one abstraction later plugged straight into Firebase push (Chapter 17) without touching the features that emit events.

9 • The feature blitz

9–11 June 2026: with the skeleton proven, features landed at a rate of several per day — because the pattern was fixed and every feature was the same recipe.

The commits tell it plainly: *UI view* → *Study Vault* → *CGPA calculator* → *Attendance tracker* → *Lost & Found* → "some more new features" (confessions, events, rides, the Q&A forum, offers, gamification followed in the same wave). Each one was the identical loop:

1. Mongoose model (with the `university` field where campus-shared);
2. a feature folder of endpoint files behind the same middleware chain;
3. a React screen talking to it through the shared axios client;
4. notification events where other users are affected;
5. seed data so it demos well.

Two features are worth singling out because they contain actual domain logic rather than CRUD: the **CGPA tracker** computes live SGPA/CGPA and answers "what SGPA do I need next semester to hit my target CGPA?" — an inverse calculation students otherwise do by trial and error; and the **attendance tracker** answers "can I skip the next class?" against the 75% detention line, which is the single most-asked question on any Indian campus.

Why the blitz worked

The speed was not heroics; it was the payoff of Part II. When the model→endpoints→screen recipe is settled and deployment is already continuous, a feature is mostly typing. The expensive part of every feature — deciding where data lives and how requests flow — had been paid once, up front.

10 · Real-time chat — the three hardest bugs

15–17 June 2026: "introducing chat with friends," followed immediately by three commits titled "socket fixes." Real-time code produced the subtlest bugs of the entire project — all three are worth understanding deeply.

10.1 Bug 1 — the silent join: an async handler race

Problem

Messages sometimes never arrived — no error, nothing in logs, the room was simply empty. The connection handler was declared `async` and did presence work first: it `await`ed the friend list and looped over friends *before* registering `socket.on("join_conversation")` and the other listeners. Socket.IO does **not** buffer events for listeners that aren't attached yet — so a client that emitted `join_conversation` immediately on connect had its join silently dropped, never entered the room, and `io.to(convId).emit(...)` broadcast to nobody. The cruel part: the browser UI usually "worked," because a human selects a conversation seconds after connecting — after the awaits had finished. Only a fast client (a fresh chat window, a test script) hit the gap.

Fix

Register **every** `socket.on(...)` handler synchronously, first thing in the connection callback; run the awaited presence work after. Verified with a test that joins in the same tick as connecting and asserts the room size reaches 2 and messages deliver both directions.

Lesson

In event-driven code, *when you subscribe* is part of correctness. Any `await` before a subscription is a race window, and races don't appear in casual manual testing — they appear for exactly the users you can't watch.

10.2 Bug 2 — the reconnect amnesia

Problem

After a network blip or server restart, Socket.IO dutifully reconnected — and chat silently went dead until the user re-selected the conversation. The reason: the client emitted `join_conversation` only once, when the chat was opened. A reconnect creates a *new server-side socket* that is in no rooms at all; the client believed it was still joined.

Fix

The socket service now tracks a set of joined conversations and re-emits `join_conversation` for all of them on every `connect` event (which fires on the initial connection *and* every reconnect), with a

matching `leaveConversation()` on chat-window cleanup. Verified by killing the backend mid-chat, restarting it, and watching a peer's message appear live without any user action.

Lesson

Reconnection restores the *transport*, never the *state*. Any state the server holds about a socket — rooms, subscriptions, presence — must be re-established by the client on every connect.

10.3 Bug 3 — the platform itself was wrong (see next chapter)

The third "socket fix" wasn't a code bug at all — it was the discovery that the hosting platform could not run *this* feature, ever. It deserves its own chapter.

11 · Leaving serverless — the move to Render

17 June 2026: "backend changed to render." The biggest infrastructure decision of the project, forced by physics rather than preference.

Problem — serverless cannot hold a WebSocket

Vercel runs functions: a request comes in, a function runs, the function dies. A WebSocket is the opposite — one connection held open for hours. Worse, the Vercel deployment was only ever serving the REST `app.js`; the Socket.IO server (`server.js`, which wraps the HTTP server) never ran there at all. Real-time chat worked locally and could **never** work in that production setup — no amount of client-side fixing would change it.

Fix — split the hosting by workload

The API moved to **Render** as a long-lived Node web service that runs the full `server.js` — Express and Socket.IO on one port, plus the node-cron reminder jobs (which also need a persistent process; serverless has nowhere to keep a schedule). The static frontend stayed on a CDN-style host, where serverless is exactly right. The trade-off knowingly accepted: Render's free tier **spins down when idle**, so the first request after a quiet period eats a cold start. That quirk later became a co-conspirator in the database bug of Chapter 12.

Lesson

Match the platform to the workload's *shape*, not its language. Stateless request/response → serverless. Long-lived connections and schedulers → a persistent process. The same app legitimately needs both at once.

12 · The database design mistake (E11000)

24 June 2026: the day-one schema decision detonates. Opening a *second* chat throws `E11000 duplicate key`. This chapter is the "realised the DB design was wrong, and fixed it properly" story

— including the twist where the first fix didn't stick.

12.1 The detonation

Problem

The first conversation always worked. The second — a different buyer on the same listing, or a second friend DM — failed with `E11000 dup key { listingId: null, participants: <userId> }`. Even the seed script quietly skipped creating a second friend DM for the demo user.

12.2 The root cause — a unique index on an array is multikey

The schema declared:

```
conversationSchema.index(  
  { listingId: 1, participants: 1 },  
  { unique: true }    // participants is an ARRAY → multikey index  
);
```

The intent was "one conversation per (listing, pair of participants)." But when an indexed field is an array, MongoDB builds a **multikey** index: one index entry per array *element*. Uniqueness is therefore enforced per (*listingId, single participant*) — meaning each user may appear in at most **one** conversation per `listingId` value. For friend DMs, where `listingId` is `null`, that caps every user at *one friend DM in total*. For the marketplace, only the first buyer per listing can ever open a chat. The constraint didn't just fail to express the intent — it enforced a different, catastrophic rule.

12.3 The first fix — and why it didn't stick

The twist

The obvious repair — drop the bad index on the Atlas cluster — worked for a few minutes, then the error *came back*. Mongoose's `autoIndex` (on by default) re-creates every index the schema declares, on every boot. Render's free tier spins down when idle, so the next login triggered a cold start, the boot re-ran `autoIndex`, and the deployed schema — which still declared the unique index — quietly rebuilt the time bomb.

The durable fix — two steps, in the right order

Step 1: change the schema and deploy it. The unique index declaration was removed and replaced with a plain, non-unique lookup index (`{ participants: 1 }`) — needed anyway for the "my conversations" query. Duplicate prevention moved to the application layer, where it already existed: `createConversation` does find-then-create per (pair + context), so the DB constraint had been redundant as well as wrong. **Step 2: drop the index once more, *after* the new code was live** — because `autoIndex` only *creates* declared indexes; it never *drops* undeclared ones. A schema change alone stops the resurrection but does not remove the corpse.

The model now carries the tombstone in a comment, so the mistake cannot be silently reintroduced by a future refactor:

```
// NOT unique: a unique index on the participants array is multikey,  
// which wrongly caps each user at one conversation per listingId value  
// - e.g. one friend DM total, or only the first buyer per listing.  
// Duplicates are prevented at the application layer instead  
// (createConversation does findOne-then-create per pair + context).  
conversationSchema.index({ participants: 1 });
```

Lessons — the most valuable in the project

- **Never put a unique index on an array field** unless per-*element* uniqueness is literally what you want. Multikey semantics are documented, surprising, and silent until the second row.
- **An index lives in two places** — the schema (intent) and the database (reality) — and they heal in one direction only: declared indexes get created; undeclared ones are never dropped. Fixing one side is half a fix.
- **Infrastructure quirks compound with data bugs.** Free-tier spin-down turned "I dropped the index" into "the index resurrects on the next login." Debugging required holding both layers in mind at once.
- **Uniqueness that involves business meaning** ("same pair + same context") often belongs in application logic, where the rule can actually be expressed — with the DB index kept for lookup speed.

13 · Production rewrites the storage plan

22 June 2026: "profile pic fixed," "uploads fixes." The original storage plan — everything to Cloudinary — met a production environment that had no Cloudinary credentials, and the plan was redesigned rather than patched.

Problem

Listing photos, lost-&-found images, avatars and UPI QR uploads worked locally but failed on the deployed backend: every image route streamed to Cloudinary, and production had no API key configured. A new profile picture also never appeared even when the save "succeeded," because the UI kept rendering the cached user object.

Fix — images move inline into MongoDB; documents keep Cloudinary

A custom Multer storage engine (`dataUrlStorage.js`) buffers the upload in memory and exposes it as a base64 **data URL** on `file.path` — a drop-in replacement, since handlers read `file.path` either way. Images now live inline in the documents they belong to (`listings.images[]` , `users.avatarUrl` , group covers, QR codes). Three guards keep documents far under MongoDB's 16 MB ceiling: client-side resize before upload, tight per-file size caps (2 MB, down from 10), and per-listing image-count

caps. The avatar screen additionally refreshes the auth context after saving, so the new picture appears everywhere immediately. **Study Vault documents (PDFs) deliberately kept the Cloudinary streaming pipeline** — they are far too large to inline, which is precisely the case Cloudinary is for.

Why inline-in-Mongo is the right call here (and when it isn't)

For small, resized images, inlining removes an entire external dependency: uploads survive redeploy, work with zero credentials, and arrive with the document in one query. The textbook objection — document bloat — is real but bounded by the caps. The moment the payload is a multi-megabyte PDF, the calculus flips, which is why the vault kept the streaming pipeline. Storage decisions are per-payload, not per-app.

14 · The login that would not stay

22–25 June 2026: "session time increases," "login fixes," "stays login fixes," "phone login fixed." Users were logged out every time they closed the app — and the investigation found two completely independent causes, one per platform.

Cause 1 — web: the app never looked

On the web, the session was never actually lost. The JWT sat safely in `localStorage` — but the start route `/` rendered the marketing landing page, which never consulted the auth context, so every visit showed "Sign in / Get started." The user *appeared* logged out because the entry point didn't check.

Cause 2 — Android: the storage genuinely vanished

On the native app, the token lived only in the WebView's `localStorage` — which Android flushes lazily and wipes when it kills the app process (swipe-close, low memory). There, the session truly disappeared.

Fix — durable storage + session-aware entry

Two shipped changes. First, an `authStorage` utility write-throughs the session to **Capacitor Preferences** (native `SharedPreferences`) alongside `localStorage`; on launch, a bootstrap step — awaited *before React renders* — copies the durable values back into `localStorage`, so the synchronous axios interceptor and socket auth never have to go async. Second, the landing and auth pages now redirect an already-authenticated user straight into the app. JWT lifetime was also set to 365 days — sessions end at logout, not on a timer, with revocation handled separately (Chapter 17).

Lesson

"Users get logged out" felt like one bug and was actually two, with different owners: a UX oversight on web and a platform storage semantics issue on Android. Debugging symptom-first ("the session is gone") would have chased the wrong layer half the time; the fix required asking, per platform, *is the token actually gone, or just unread?*

15 · Going native — the Android app

22–26 June 2026: the same codebase became an installable Android app. Almost every "mobile bug" turned out to be a web bug with better camouflage.

Capacitor 8 wraps the built web app (`frontend/dist`) in a native shell with a Gradle project under `frontend/android/`. Native HTTP means the app calls the API without any CORS involvement at all. The build chain is scripted (`npm run android` → build web → sync → open Android Studio); launcher icons and splash screens are generated from the MANIT crest by a script.

Practical potholes, as encountered: Capacitor 8 requires **JDK 21** — a machine whose `PATH` points at JDK 17 fails with confusing Gradle errors, and the working answer was Android Studio's bundled JBR. Release signing reads `keystore.properties` with a git-ignored keystore. And an early convenience — committing the built APK into the repo — was reversed within days (*"Delete Manithub.apk"*): binaries bloat every future clone forever, and releases belong in release channels, not history. One real limitation was accepted with eyes open: with no over-the-air update layer, every JS change reaching APK users requires a rebuild — fine at this scale, and the web app is always current.

16 · The restructures

Twice — 17 June and 13 July — the project stopped adding features and reorganised itself. Both were done as proper branch-and-merge work (the June one is the repo's PR #1), not loose commits.

The June restructure lifted the backend out of a nested `src/` into the flat layout this document has been describing: `server.js` and `app.js` at the backend root, with `config/`, `middleware/`, `models/`, `socket/`, `jobs/`, `utils/`, and the 23 feature folders beside them. The frontend was restructured in the same sweep, and the dead Vercel-backend configuration was removed once Render was live. The July restructure was a smaller tidy-up in the same spirit.

Why restructure mid-project instead of pushing on?

Because the nested layout was actively lying: the entry point Vercel deployed and the entry point that ran locally were different files (Chapter 11's discovery), and paths no longer matched what the docs and configs implied. A structure that misleads is worse than churn. Doing it as an isolated PR kept the risky, everything-moves change reviewable and revertible — worth it even solo.

17 · Security hardening & email verification

4 July ("Security Increases") and 14 July 2026 ("email authentication"): with the feature set stable, a dedicated pass took the app from "works" to "defensible." The philosophy: defence in depth — no single control trusted alone.

17.1 Accounts and sessions

- **Passwords:** bcrypt with salt, stored with `select:false` so no query can leak the hash by accident; minimum 8 characters enforced.
- **Revocable long sessions:** the 365-day JWT (Chapter 14) gets a `tokenVersion` claim; a password reset increments the user's version, instantly invalidating *every* previously issued token. Long, friction-free sessions and revoke-on-compromise — usually presented as a trade-off — coexist with one integer.
- **Password reset:** a random token whose **SHA-256 hash** alone is stored (30-minute expiry) and emailed via Nodemailer; the raw token appears in an API response only in development with no SMTP configured.
- **Email verification (the 14 July finale):** signup issues a 6-digit code — stored hashed, 30-minute expiry — and login returns 403 until it is confirmed, proving the `@stu.manit.ac.in` address is genuinely owned. Deliberately **config-gated**: with no SMTP configured (local dev), signup falls back to the old flow instead of breaking.
- **Rate limiting:** 600 requests/15 min per IP across the API; on auth routes, 20 *failed* attempts per 15 minutes (successful logins don't count) — brute force, signup spam and code-guessing all hit the same wall.

17.2 Boundaries

- **Tenancy re-checked everywhere:** the socket layer re-verifies university membership before any join or send — REST middleware alone isn't enough when a second transport exists.
- **CORS matched precisely:** preview deployments are matched on the *full project slug as a subdomain label*, closing the hole where any attacker-registered `manithub-*.vercel.app` would have passed a loose substring check.
- **Uploads:** raster-only MIME allow-list — **SVG rejected outright**, since SVG is a script container that can carry XSS into any page that renders it.

17.3 Content moderation — three layers

1. **Text filter at the gate:** a self-contained English + Hindi/Hinglish profanity filter that de-obfuscates leetspeak, spacing and character repeats, with whole-word matching to avoid false positives, applied to public content (confessions, forum, listings). Private DMs are deliberately

not filtered — private conversation is not the app's to police; reporting covers abuse there.

Extensible at runtime via an env var, so new slang never needs a deploy.

2. **Reports** → **auto-hide** → **strikes** → **ban**: content auto-hides after 3 distinct reports and the author takes a strike; 3 strikes auto-suspends the account, enforced at login, API *and* socket.

Admins keep a manual review queue that overrides the automation in both directions.

3. **Image moderation, env-gated and fail-open**: an optional hook (Google Vision SafeSearch or Sightengine) rejects explicit images *before storage*. Off by default; on a provider outage it fails open, because blocking every campus upload over a third-party blip is worse than letting the report queue backstop a rare miss.

Why "config-gated" recurs so often

Email verification, push notifications, and image moderation all switch on only when their credentials exist — and degrade gracefully when they don't. That is what lets one codebase serve local dev (no SMTP, no Firebase, no Vision key) and production without forks or mocks. Every external dependency is an upgrade, never a requirement.

18 · The frontend, as it ended up

The frontend was never "written last" — it grew screen-by-screen behind each feature. But it was *polished* last, deliberately, once the product underneath had stopped moving.

The structure mirrors the backend's discipline: `api/` holds one axios client per feature (all sharing an interceptor that attaches the JWT and unwraps the response envelope); `screens/` has one route-level component per page; `components/` holds the UI kit; `context/` carries Auth and Theme providers; `utils/` contains the socket service (with the reconnect-re-join logic of Chapter 10) and the durable auth storage of Chapter 14.

The design system is the MANIT identity — navy, crimson and gold tokens over Tailwind — with full dark mode, a ⌘K command palette, and navigation reorganised from a flat screen-list into four hubs (Academics · Study · Marketplace · Campus Life) with in-page tabs. That reorganisation happened for an honest reason: at 20+ features, the original sidebar had become a wall; the information architecture had to grow up along with the feature count.

Why invest in "premium feel" for a student project?

Because adoption is the product's only metric. A campus app competes with WhatsApp's zero-friction inertia; students only switch to something that feels *better*, not merely functional. Dark mode, motion, and a command palette are not decoration here — they are the argument for switching.

19 · Lessons learned

If the whole story compresses to ten lines, these are the lines.

1. **Deploy on day one.** Fourteen deployment-fix commits against an empty app cost nothing; the same lessons against live users would have been incidents.
2. **Design the data first;** screens are cheap to change, collections are not.
3. **Never put a unique index on an array field.** Multikey semantics enforce per-element uniqueness — a different rule than intended, silent until the second row.
4. **Indexes live in the schema *and* the database,** and they heal in one direction only. A durable index fix = schema change, deploy, *then* a one-time drop.
5. **Match hosting to workload shape:** serverless for request/response, a persistent process for WebSockets and cron. One app can — and here does — need both.

6. **Register event handlers before any `await`**. Subscription timing is correctness; races hide from manual testing by design.
7. **Reconnection restores the transport, not the state.** The client must re-assert rooms and subscriptions on every connect.
8. **When production lacks a dependency, redesign rather than patch** — inlining images removed a whole external service; the PDF pipeline kept it where it earned its place.
9. **One symptom can be two bugs.** "Users get logged out" was a web UX oversight *and* an Android storage semantics issue; per-platform diagnosis found both.
10. **Gate every external service on config and degrade gracefully.** That is what keeps one codebase honest across dev and production — and what let security land in layers instead of one big bang.

20 · Appendix — the full timeline

The whole story again, at commit-log resolution. Dates are commit dates on `main`.

19 May	Day one. Initial commit — and 13 more the same day: dependency fixes, root route, CORS origin, Vite env-var naming, Atlas connection, the serverless Mongo connection gate, hardened production config. The deployment war (Ch. 6).
20 May	Campus maps & the name. Exact MANIT boundary (KML), embedded custom map, map dashboard redesign; Netlify SPA routing fixed; project renamed Manit Hub (Ch. 7).
4 Jun	Feeling alive. Frontend iteration, <code>vercel.json</code> , seed data, notification fixes, all 41 campus locations integrated (Ch. 8).
9–11 Jun	The feature blitz. App integration, UI view, Study Vault, CGPA calculator, Attendance tracker, Lost & Found, "some more new features" — confessions, events, rides, forum, offers, gamification (Ch. 9).
15–17 Jun	Chat, and the socket wars. Chat with friends ships; three rounds of socket fixes — the async-handler join race and reconnect re-join (Ch. 10); the discovery that serverless can't hold WebSockets → backend moves to Render ; the big backend/frontend restructure lands as PR #1 (Ch. 11, 16).
22 Jun	Production teaches storage & sessions. APK fixes; frontend moves Netlify → Vercel; session lifetime to 365 days; login fixes; profile pictures and uploads move to inline data-URL storage (Ch. 13, 14).
24–25 Jun	The E11000 day. The multikey unique-index bug is diagnosed, the schema fixed (<code>Conversation.js</code>), the autoIndex-resurrection twist understood, the durable fix deployed; stay-logged-in and phone login finished (Ch. 12, 14).

26 Jun	Repo hygiene. The committed APK is deleted from the repository; distribution moves to a release link (Ch. 15).
3 Jul	Telling the story. A full video tour is produced and linked from the landing page and README.
4 Jul	"Security Increases." Rate limiting, tokenVersion revocation, precise CORS slug-matching, mass-assignment whitelists, SVG rejection, the EN+Hinglish profanity filter, report→auto-hide→strike→ban pipeline, env-gated image moderation (Ch. 17).
13 Jul	Second restructure. A final layout tidy-up (Ch. 16).
14 Jul	"Email authentication." The 6-digit signup OTP flow ships — hashed codes, 30-minute expiry, 403-until-verified, config-gated on SMTP (Ch. 17). The story, for now, is up to date.

End of the Build Story. Companion documents: *Manit-Hub-Developer-Notes.pdf* (the function-by-function study guide) and the README (setup & API reference).

Built by students, for students. 🎓 — Not an official institute portal.